🌌 Cosmic Anomaly Hunter

Project Summary Report

1. Project Overview

Cosmic Anomaly Hunter is an interactive AI-driven tool designed to assist astronomers in identifying interesting or unusual structures in large astronomical image archives. The system combines **supervised classification** (galaxy morphology), **unsupervised anomaly detection**, and **explainability** (Grad-CAM) into a user-friendly Streamlit web application.

The central research question is:

Can AI help astronomers identify interesting or unusual structures in large astronomical image archives that could otherwise be overlooked?

The project demonstrates a complete end-to-end pipeline from raw data ingestion to interactive visual exploration.

2. Data Acquisition & Preparation

We used the **Galaxy Zoo 2 (GZ2)** dataset, which provides volunteer classifications for hundreds of thousands of SDSS galaxies.

Data files:

- zoo2MainSpecz.csv – Contains morphological vote fractions (e.g., t01_smooth_or_features_a01_smooth_fraction, t04_spiral_a08_spiral_fraction, t08_odd_feature_a24_merger_fraction).
- gz2_filename_mapping.csv – Maps SDSS dr7objid to image asset_id (used for file naming).
- **Image files** – Supplied as .jpg images named by asset_id (e.g., 1.jpg, 2.jpg, …).

Data preparation steps:

1. Label assignment – We defined three classes:

1. **0** – Elliptical / Smooth (if smooth_fraction > 0.5).
2. **1** – Spiral (if spiral_fraction > 0.5).
3. **2** – Merger / Other (if merger_fraction > 0.5 or fallback).

2. **Merging** – Joined the classification data with the filename mapping on dr7objid.
3. **Image existence check** – Verified that all matched images were present; missing entries were dropped.
4. **Final metadata** – Saved as metadata.csv with columns: filename, dr7objid, asset_id, label.

Dataset Summary:

• **Final dataset size:** ~50,000 images (exact count depends on the downloaded subset)
• Class distribution:
  • **0 (Elliptical):** ~40%
  • **1 (Spiral):** ~50%
  • **2 (Merger/Other):** ~10%

3. AI / Data Science Components

3.1 Classifier (Identify Mode)

- **Architecture:** ResNet50 pre-trained on ImageNet, fine-tuned for 3-class classification.
- **Training:** Adam optimizer, learning rate 1×10−4, batch size 32, 3 epochs.
- **Performance:** Achieved **~95% validation accuracy** after 3 epochs.
- **Output:** Predicted class (Elliptical/Spiral/Merger) and confidence score.

3.2 Anomaly Detection (Discover Mode)

• **Approach:** Convolutional Autoencoder trained **only on normal galaxies** (labels 0 and 1).
• **Architecture:** 5-layer encoder / 5-layer decoder with transposed convolutions.
• Anomaly score: Reconstruction Mean Squared Error (MSE) between input and reconstructed image.
  • High MSE = unusual object (potential merger, lens, or artifact).
• **Training:** 20 epochs, MSE loss, Adam optimizer.
• **Result:** The autoencoder reconstructs normal galaxies well, but poorly reconstructs mergers/ others, thus flagging them as anomalies.

3.3 Explainability (Explore Mode)

- **Method:** Grad-CAM (Gradient-weighted Class Activation Mapping).
- **Implementation:** Hooked the last convolutional layer of ResNet50 to obtain gradients and activations; generated a heatmap overlay highlighting the regions most influential for the predicted class.
- **Benefit:** Provides visual interpretability, allowing users to see *why* the model made a certain prediction.

3.4 Dimensionality Reduction (Planned)

- **Technique:** t-SNE on feature embeddings from the classifier's penultimate layer.
- **Purpose:** To create a 2D map where similar galaxies cluster together, aiding in visual exploration and discovery of outliers.

4. System Architecture & Workflow

The end-to-end pipeline is implemented in Python and deployed as a **Streamlit web application**.

Workflow:

1. **User uploads** an astronomical image (or the app loads a sample from the dataset).
2. **Image preprocessing** – Resize to 224×224, normalize with ImageNet statistics.
3. **Classifier forward pass** – Obtains class probabilities and confidence.
4. **Autoencoder reconstruction** – Computes MSE anomaly score.
5. **Grad-CAM computation** – Generates heatmap for the predicted class.
6. **Display** – Shows original image, heatmap overlay, prediction, confidence, and anomaly score side-by-side.

Technology stack:

- **Deep Learning:** PyTorch, torchvision.
- **Data handling:** Pandas, NumPy, PIL.
- **Visualization:** Matplotlib, Streamlit.
- **Explainability:** Custom Grad-CAM implementation with OpenCV.

5. Application Features

The Streamlit app provides:

- **Upload** – Drag-and-drop or browse for JPG/PNG/JPEG images.
- **Sample fallback** – If no image is uploaded, the first image from the metadata is used.
- **Side-by-side view** – Original image on the left, Grad-CAM heatmap overlay on the right.
- **Predictions** – Displays the predicted class (e.g., "Elliptical") with confidence percentage.
- **Anomaly Score** – A numerical score; higher values indicate more unusual structures.

Example output:

- Prediction: **Elliptical (confidence: 99.12%)**
- Anomaly Score: **3492.19** (higher = more unusual)

6. Results & Preliminary Evaluation

- The classifier achieves high accuracy on validation data, indicating that the fine-tuned ResNet50 can reliably distinguish between elliptical, spiral, and merger galaxies.
- The autoencoder successfully assigns higher reconstruction errors to galaxies labelled as "Merger/Other" compared to normal galaxies, validating its anomaly-detection capability.
- Grad-CAM heatmaps qualitatively align with human intuition (e.g., focusing on spiral arms for spiral galaxies, or on the central bulge for ellipticals).

7. Discussion & Strengths

- **Combines supervised and unsupervised learning** – Demonstrates versatility in applying both paradigms to a real-world problem.
- **Explainable AI** – Grad-CAM opens the black box, building trust with end-users.
- **Interactive interface** – Makes the tool accessible to non-experts (astronomers, educators, citizen scientists).
- **Scalability** – The pipeline can be extended to larger datasets (e.g., full GZ2, Hubble, JWST).
- **Portfolio appeal** – Visually striking, scientifically relevant, and technically comprehensive.

Challenges encountered:

- Image file naming discrepancies – resolved by using asset_id from the mapping file.
- Streamlit deprecation warnings – updated use_column_width to use_container_width.
- Grad-CAM hook warnings – non-full backward hooks, but functionality works.

8. Future Work & Enhancements

- **Bounding box detection** – Incorporate an object detector (e.g., YOLO) to locate multiple

galaxies in a single field.
- **Multi-band imagery** – Support FITS files and colour composites from SDSS/HST.
- **Gravitational lens candidate search** – Fine-tune the anomaly detector specifically for arc-like features.
- **Active learning** – Integrate human feedback to improve the anomaly ranking.
- **Deployment** – Host the app on a cloud platform (e.g., Streamlit Cloud, Hugging Face Spaces) for public use.
- **Embedding map** – Add a t-SNE visualization to browse the entire dataset and spot outliers interactively.

9. Conclusion

The **Cosmic Anomaly Hunter** successfully demonstrates how modern AI techniques can assist astronomers in sifting through massive image archives. By combining classification, anomaly detection, and explainability in an interactive dashboard, the tool not only performs accurate predictions but also provides visual justifications and flags potentially interesting candidates for human follow-up.

This project stands as a strong portfolio piece, showcasing skills in:

- Data engineering (cleaning, merging, verifying).
- Deep learning (transfer learning, autoencoders).
- Explainable AI (Grad-CAM).
- Full-stack development (Streamlit, PyTorch).

The project tagline – *"Let AI search the universe for what humans might have missed"* – captures the essence of its scientific and exploratory value.

10. Project Repository & Files – Detailed Breakdown

All source code, data preparation scripts, model training, and the app are organised in the following structure:

Plaintext

Cosmic hunter/

```
├── data/
│   ├── zoo2MainSpecz.csv          # Raw GZ2 vote fractions
│   ├── gz2_filename_mapping.csv   # Raw ID-to-filename mapping
│   └── metadata.csv               # Curated master file (generated)
├── images_gz2/images/             # All .jpg galaxy images
├── models/
│   ├── classifier_resnet50.pth    # Trained classifier weights
│   └── autoencoder.pth            # Trained autoencoder weights
└── src/
    ├── prepare_data.py            # Data preparation & merging
    ├── data_loader.py             # PyTorch Dataset & transforms
    ├── classifier.py              # Trains the ResNet50 classifier
    ├── autoencoder.py             # Trains the anomaly-detection AE
    ├── explain.py                 # Grad-CAM implementation
    ├── embedding.py               # (Planned) t-SNE embedding
    └── app.py                     # Streamlit interactive frontend
```

□ □ requirements.txt # Python dependencies

10.1 Data Files (CSVs)

File	Role	Consumed By
zoo2MainSpecz.csv	Raw classification data from Galaxy Zoo 2. Contains volunteer vote fractions for morphological features, e.g. t01_smooth_or_features_a01_smooth_fraction(smoothness), t04_spiral_a08_spiral_fraction (spiral structure), and t08_odd_feature_a24_merger_fraction (merger signature).	prepare_data.py
gz2_filename_mapping.csv	Raw mapping file. Links the SDSS object ID (dr7objid) to a numerical asset_id(used for the actual image filenames, e.g. 1.jpg, 2.jpg).	prepare_data.py
metadata.csv	**Generated master file** (output of prepare_data.py). Contains only the images that were successfully found on disk. Columns: filename (e.g. 1.jpg), dr7objid (SDSS ID), asset_id (numeric ID), and label (0 = Elliptical, 1 = Spiral, 2 = Merger/Other). This is the **single source of truth** used by all ML scripts and the app.	

data_loader.py (and indirectly all training/inference scripts)

10.2 Python Scripts – Detailed Working

prepare_data.py – Data Preparation

• **Inputs**: zoo2MainSpecz.csv, gz2_filename_mapping.csv, and the raw image folder.
• Function:
    •
    • Applies a label assignment rule:

*

3. Merges the classification data with the filename mapping on dr7objid.
4. Constructs the actual image filename from asset_id (e.g., 1.jpg).
5. **Verifies image existence** – drops entries whose image files are missing.
6. **Saves** the curated, validated subset as metadata.csv.

- **Output**: metadata.csv.

data_loader.py – Dataset & Transforms

• **Inputs**: metadata.csv path and image directory path.
• Function:
    • Defines the GalaxyDataset class (inherits from torch.utils.data.Dataset).
    • Reads metadata.csv during initialisation.
    • In __getitem__, it loads the image from disk using the filename column, converts it to RGB, applies the specified transforms, and returns (image_tensor, label).
    • Provides the get_transforms() function that standardises images to 224×224, converts them to PyTorch tensors, and normalises them using ImageNet statistics (mean & std).
• **Role**: Serves as the **data bridge** between the CSV metadata and the neural network models.

classifier.py – Supervised Training (Identify Mode)

• **Inputs**: metadata.csv and the image folder (via data_loader.py).
• Function:

    • Splits the dataset into **80% training** and **20% validation**.
    • Loads a **ResNet50** pre-trained on ImageNet.
    • Replaces the final fully-connected layer with a new one for **3 classes**.
    • Trains the network using **CrossEntropyLoss** and the **Adam** optimizer for 3 epochs.
    • Evaluates validation accuracy after each epoch.
    • **Saves** the trained model weights to models/classifier_resnet50.pth.
• **Output**: classifier_resnet50.pth (model weights).

autoencoder.py – Unsupervised Anomaly Detection (Discover Mode)

• **Inputs**: metadata.csv and the image folder (via data_loader.py).
• Function:
    • **Filters** the dataset to keep **only normal galaxies** (labels 0 and 1). Mergers (label 2) are excluded from training.
    • Defines a **convolutional autoencoder** (5-layer encoder, 5-layer decoder with transposed convolutions).
    • Trains the autoencoder using **Mean Squared Error (MSE) loss** – it learns to reconstruct only normal galaxies.
    • **Saves** the trained autoencoder weights to models/autoencoder.pth.
• **Output**: autoencoder.pth (model weights).
• **Anomaly logic**: During inference, the reconstruction error (MSE) is used as the anomaly score. High error = unusual structure (because the model never learned to reconstruct it well).

explain.py – Grad-CAM Explainability (Explore Mode)

• **Inputs**: The trained classifier (classifier_resnet50.pth loaded into a ResNet50 model) and an input image tensor.
• Function:
    • Hooks into the **last convolutional layer** of ResNet50 (via forward/backward hooks).
    • Performs a forward pass to get the class prediction, then a backward pass to get gradients for the predicted class.
    • Computes a **Grad-CAM** heatmap by globally pooling the gradients and weighting the convolutional feature maps.
    • Upsamples the heatmap to 224×224, applies ReLU, and normalises it to a [0,1] range.
• **Output**: A 2D NumPy array representing the heatmap, which is overlaid on the original image in the app to show which pixels most influenced the prediction.

embedding.py – Dimensionality Reduction (Planned Enhancement)

- **Status**: Currently a stub; fully developed for future integration.
- **Planned function**: Extract high-dimensional feature vectors from the classifier's penultimate layer, then apply **t-SNE** or **UMAP** to project them into 2D. This will allow users to visually browse the entire dataset, spot natural clusters, and identify outlier galaxies on the embedding map.

app.py – Interactive Web Interface

• **Inputs**: The trained models (.pth files), metadata.csv, and user-uploaded images.
• Function:
  • **Caches** and loads the classifier and autoencoder using @st.cache_resource for performance.
  • Displays an **uploader** for JPG/PNG/JPEG images; if none is uploaded, it loads a sample image from metadata.csv.
  • Preprocesses the image using the same transforms from data_loader.py.
  • Runs **classifier inference** to obtain the predicted class and confidence.
  • Runs **autoencoder inference** to compute the reconstruction error (anomaly score).
  • Calls grad_cam() from explain.py to generate the heatmap.
  • Uses **Streamlit columns** to display the original image and the Grad-CAM overlay side-by-side.
  • Renders the prediction, confidence, and anomaly score as text below the images.
• **Role**: Orchestrates the entire pipeline and serves as the **front-end** for interactive exploration.

10.3 Model Files (.pth)

classifier_resnet50.pth:- Saved weights of the fine-tuned ResNet50 classifier. Loaded by app.py to perform real-time image classification.

autoencoder.pth:- Saved weights of the trained convolutional autoencoder. Loaded by app.py to compute the anomaly score for any input image.

10.4 Requirements & Execution

requirements.txt – Python dependencies:

Plaintext

torch

torchvision

streamlit

pandas

numpy

scikit-learn

matplotlib

opencv-python-headless

pillow

tqdm

Installation Command:

Bash

pip install -r requirements.txt

Running the Full Pipeline

1. **Prepare the data:**

   python src/prepare_data.py
2. **Train the classifier:**

python src/classifier.py
3. **Train the autoencoder:**

python src/autoencoder.py
4. **Launch the app:**

streamlit run src/app.py